PORTAL DE ALERTAS PARA LIVES

Documento de Requisitos - V1

Projeto de estudo prático de desenvolvimento de software

Versão	1.0
Data de início	17 de setembro de 2026
Prazo da V1	16 de outubro de 2026
Objetivo	Aplicação funcional de ponta a ponta, sem foco comercial
Escopo de deploy	Fora da V1; será decidido posteriormente

Premissa: o objetivo principal é aprender construindo algo real, demonstrável e integrado.

1. Contexto e objetivo

O cliente deseja uma aplicação web para criadores de conteúdo configurarem alertas visuais que possam aparecer durante transmissões ao vivo. A V1 será usada como exercício prático de engenharia de software e poderá ser testada inicialmente com um canal pequeno, inclusive um canal do próprio cliente.

O objetivo não é competir comercialmente com plataformas existentes, nem validar monetização. O objetivo é ter uma solução funcional de ponta a ponta, com autenticação, integração com YouTube, configuração de alertas, simulação de eventos e execução de animações em um overlay consumível por software de transmissão.

Definição de sucesso da V1

Uma pessoa deve conseguir entrar com Google, conectar/identificar seu canal do YouTube, configurar um alerta a partir de um template, abrir um overlay em outra janela, disparar um evento de teste e ver a animação aparecer corretamente. A configuração deve permanecer salva entre sessões.

2. Visão do produto

O fluxo conceitual esperado é:

EVENTO → REGRA DE ALERTA → TEMPLATE CONFIGURADO → OVERLAY

Exemplo de uso: um novo inscrito é detectado; o sistema identifica a configuração correspondente; aplica os dados do evento ao template; e exibe a animação no overlay aberto durante a live.

3. Escopo funcional da V1

A V1 deve contemplar os seguintes blocos funcionais:

- Autenticação com conta Google.
- Identificação ou conexão do canal do YouTube do usuário autenticado.
- Dashboard simples com estado da integração e acesso às configurações.
- Configuração de alertas por tipo de evento.
- Seleção de templates de animação predefinidos.
- Customização simples de cada alerta.
- Simulador interno de eventos para desenvolvimento e demonstração.
- Overlay acessível por URL própria e capaz de receber/disparar animações.
- Integração real com ao menos um evento observável do YouTube.
- Persistência das configurações do usuário.
- Tratamento básico de erros e estados de integração.

4. Requisitos funcionais

4.1 Autenticação e conta

RF-01 - Login com Google: o usuário deve conseguir entrar na aplicação utilizando sua conta Google.

RF-02 - Sessão: após o login, o usuário deve permanecer autenticado de forma segura e conseguir encerrar a sessão.

RF-03 - Dados mínimos: a aplicação deve exibir apenas os dados de perfil necessários para identificação do usuário e do canal conectado.

RF-04 - Reconexão: caso a autorização do Google/YouTube expire, seja revogada ou falhe, a aplicação deve informar o problema e permitir nova autorização.

4.2 Integração com YouTube

RF-05 - Canal autenticado: após a autorização adequada, a aplicação deve identificar o canal do YouTube associado ao usuário e apresentar informações básicas do canal.

RF-06 - Estado da integração: o dashboard deve indicar de forma clara se o YouTube está conectado, desconectado ou com erro de autorização.

RF-07 - Eventos externos: o sistema deve possuir um mecanismo para consumir eventos ou mudanças disponibilizadas pelas APIs oficiais do YouTube e convertê-los para eventos internos da aplicação.

RF-08 - Novo inscrito: o domínio da aplicação deve suportar o evento NEW_SUBSCRIBER e permitir sua execução tanto por integração real quanto pelo simulador.

RF-09 - Novo membro: o domínio da aplicação deve suportar o evento NEW_MEMBER. A integração real deve ser tentada apenas quando a conta e as permissões do YouTube permitirem. Caso o acesso oficial seja indisponível, a limitação deverá ser documentada e o fluxo completo deverá ser demonstrado pelo simulador.

RF-10 - Evento adicional testável: para facilitar testes reais sem depender continuamente de novas inscrições ou memberships, a V1 deve suportar ao menos um evento adicional fácil de provocar no YouTube, preferencialmente relacionado ao chat de uma live de teste.

Observação de produto: o cliente não exige uma técnica específica para captura de eventos. Webhook, polling, server-streaming ou outro mecanismo suportado oficialmente pode ser utilizado conforme a capacidade da API. O requisito é a integração funcional e bem encapsulada.

4.3 Alertas e templates

RF-11 - Tipos de alerta: cada tipo de evento suportado deve possuir sua própria configuração de alerta.

RF-12 - Templates: o usuário deve escolher entre pelo menos 3 templates visuais predefinidos. Não é necessário um editor livre no estilo Canva.

RF-13 - Customização: cada alerta deve permitir customizar, no mínimo: texto, duração e habilitado/desabilitado. Como diferenciais opcionais, podem ser incluídos som, imagem, cor ou outras propriedades simples.

RF-14 - Variáveis do evento: o texto do alerta deve aceitar dados dinâmicos do evento, como nome do usuário, quando essa informação estiver disponível.

RF-15 - Pré-visualização: o usuário deve conseguir visualizar/testar a animação antes de usá-la no overlay.

RF-16 - Persistência: as configurações salvas devem continuar disponíveis após logout/login ou reinício da aplicação.

4.4 Simulador de eventos

RF-17 - Simulação manual: a aplicação deve possuir uma área explícita de Simular evento, disponível para desenvolvimento e demonstração.

RF-18 - Dados simulados: o usuário deve poder escolher o tipo do evento e fornecer dados de exemplo relevantes, como nome, avatar ou mensagem, quando aplicável.

RF-19 - Mesmo pipeline: um evento simulado deve percorrer o mesmo fluxo interno de um evento real a partir do momento em que é recebido pelo sistema.

RF-20 - Disparo no overlay: ao disparar um evento pelo simulador, qualquer overlay ativo do usuário deve exibir a animação correspondente.

4.5 Overlay

RF-21 - URL do overlay: cada usuário deve possuir uma URL de overlay adequada para uso como Browser Source ou equivalente em software de transmissão.

RF-22 - Execução independente: o overlay deve poder permanecer aberto independentemente do painel de configuração.

RF-23 - Atualização em tempo de execução: quando um evento for recebido, o overlay ativo deve apresentar o alerta sem exigir recarregamento manual da página.

RF-24 - Fila mínima: caso dois ou mais eventos sejam disparados em sequência rápida, o sistema deve evitar sobreposição destrutiva. Uma fila simples ou comportamento equivalente é aceitável.

RF-25 - Dados sensíveis: a URL e o código executado no overlay não devem expor credenciais OAuth, segredos de integração ou outros dados sensíveis.

4.6 Dashboard e usabilidade

RF-26 - Dashboard: o painel inicial deve mostrar de forma simples o canal conectado, o estado da integração e atalhos para alertas, simulador e overlay.

RF-27 - Feedback: ações como salvar, conectar, desconectar, testar e falhas de integração devem gerar feedback visual compreensível.

RF-28 - Estados vazios: quando não houver canal conectado, configuração criada ou live ativa, a interface deve orientar o usuário sobre o próximo passo.

5. Requisitos de arquitetura e qualidade do exercício

O cliente não determina linguagem, framework, banco de dados, biblioteca de interface ou provedor de nuvem. Entretanto, para esta V1, ficam estabelecidas as seguintes restrições de arquitetura do exercício:

- A solução deve ser tratada como uma aplicação monolítica, sem necessidade de microserviços.
- A aplicação deve possuir uma API REST como interface principal entre front-end e back-end.
- Dados de usuário, integrações e configurações de alertas devem ser persistidos em banco de dados.
- Integrações externas devem ser isoladas do restante das regras de negócio, permitindo que eventos simulados e reais usem o mesmo domínio interno.
- Credenciais, tokens e segredos devem ser mantidos no lado servidor e tratados de forma segura.

- O código deve possuir tratamento de erros, validação de entrada e logs suficientes para investigar falhas.
- Fluxos críticos devem possuir testes automatizados em nível compatível com o projeto, especialmente regras de evento/alerta e endpoints principais.
- A solução deve possuir instruções claras para execução local por outra pessoa.

Não é requisito da V1

Escolher AWS, Azure, GCP ou qualquer outra infraestrutura. A aplicação pode rodar localmente durante o desenvolvimento e a decisão de deploy será feita depois.

6. Modelo conceitual esperado

Sem impor um modelo físico de banco de dados, espera-se que a solução represente conceitos equivalentes a:

Conceito	Responsabilidade
Usuário	pessoa autenticada via Google.
Canal conectado	canal do YouTube associado ao usuário e estado da autorização.
Tipo de evento	ex.: NEW_SUBSCRIBER, NEW_MEMBER, LIVE_CHAT_MESSAGE.
Template	definição visual reutilizável de uma animação.
Configuração de alerta	relação entre evento, template e customizações do usuário.
Evento	ocorrência real ou simulada com dados associados.
Overlay	superfície que recebe eventos e apresenta as animações durante a live.

7. Regras de negócio principais

RN-01 - Regra: Um evento deve ter um tipo conhecido pelo sistema para poder acionar um alerta.

RN-02 - Regra: Se não houver alerta habilitado para o tipo de evento recebido, nada deve ser exibido no overlay.

RN-03 - Regra: Eventos simulados e eventos reais devem produzir o mesmo comportamento visual quando possuem o mesmo tipo e os mesmos dados.

RN-04 - Regra: Cada usuário deve visualizar e alterar somente suas próprias configurações e integrações.

RN-05 - Regra: Dados dinâmicos indisponíveis não devem quebrar o alerta; o template deve lidar com valores ausentes de forma previsível.

RN-06 - Regra: O sistema não deve considerar a integração de memberships obrigatoriamente disponível, pois ela depende das permissões do canal e da API oficial.

RN-07 - Regra: A integração de novos inscritos deve respeitar as limitações de visibilidade e disponibilidade impostas pelo YouTube.

8. Critérios de aceite da V1

A entrega será considerada funcional quando os cenários abaixo puderem ser demonstrados sem alteração manual de código durante a apresentação:

CA-01 Entrar no sistema usando uma conta Google.

CA-02 Visualizar o canal do YouTube conectado ou concluir o fluxo necessário para conectá-lo.

CA-03 Criar/editar a configuração de alerta de Novo inscrito escolhendo um dos templates disponíveis.

CA-04 Salvar a configuração e confirmar que ela permanece após nova sessão.

CA-05 Abrir a URL do overlay em uma segunda janela.

CA-06 Usar o simulador para disparar NEW_SUBSCRIBER e visualizar a animação no overlay.

CA-07 Usar o simulador para disparar NEW_MEMBER e visualizar a animação correspondente.

CA-08 Demonstrar pelo menos um evento real e fácil de gerar no YouTube, como uma mensagem no chat de uma live de teste, chegando ao sistema e acionando um comportamento configurado.

CA-09 Demonstrar a integração de inscritos reais quando a API fornecer dados observáveis; caso isso não seja possível no momento do teste, apresentar logs/dados da integração e explicar a limitação oficial.

CA-10 Provocar um erro previsível de integração/autorização e mostrar que a aplicação apresenta feedback compreensível sem quebrar o restante do sistema.

CA-11 Abrir o projeto em ambiente local seguindo somente a documentação fornecida no repositório.

9. Fora do escopo da V1

- Deploy em ambiente de produção e definição de infraestrutura cloud.
- Cobrança, planos, assinatura ou qualquer mecanismo de monetização.
- Marketplace de templates.
- Integrações com Twitch, Kick, TikTok, PIX, Streamlabs ou outras plataformas.
- Editor visual completamente livre ou drag-and-drop no estilo Canva.
- Aplicativo mobile nativo.
- Times, múltiplos usuários por canal, permissões avançadas ou painel administrativo complexo.
- Analytics avançado, relatórios históricos ou métricas comerciais.
- Alta disponibilidade, auto-scaling ou arquitetura distribuída.
- Biblioteca extensa de animações. Três templates bons são suficientes para a V1.

10. Melhorias opcionais (não bloqueiam a entrega)

- Som por alerta e controle de volume.
- Avatar/imagem dinâmica do usuário que gerou o evento.
- Editor simples de cores, posição ou tamanho.
- Tela de histórico dos últimos eventos recebidos.
- Botão para copiar a URL do overlay.
- Fila visual de eventos pendentes.

- Modo de demonstração com dados fictícios pré-prontos.
- Reconexão automática do overlay após perda temporária de comunicação.

11. Prazo e checkpoints

Prazo proposto para a V1: 4 semanas, com entrega final em sexta-feira, 16 de outubro de 2026. O foco é aprender e chegar a uma demonstração funcional; não se espera acabamento de produto comercial.

Checkpoint	Data alvo	Resultado observável
CP-1	25/09/2026	Login Google, sessão e canal identificado/conectado.
CP-2	02/10/2026	Alertas configuráveis, 3 templates e simulador funcionando localmente.
CP-3	09/10/2026	Overlay por URL recebendo eventos em tempo de execução, com fila/comportamento mínimo para concorrência.
CP-4 - V1	16/10/2026	Integração YouTube demonstrável, critérios de aceite principais atendidos, testes essenciais e documentação de execução.

Regra do exercício: se uma integração externa impedir a conclusão por restrição oficial de acesso, o desenvolvedor deve documentar a limitação, manter o domínio e o simulador funcionando e demonstrar evidências da tentativa de integração. Isso não autoriza substituir silenciosamente o requisito por mock.

12. Roteiro sugerido para a apresentação final

1. Abrir a aplicação sem sessão e entrar com Google.
2. Mostrar o canal conectado e o estado da integração.
3. Abrir a configuração de Novo inscrito, selecionar um template e alterar o texto.
4. Abrir o overlay em uma segunda janela.
5. Disparar um inscrito pelo simulador e mostrar a animação no overlay.
6. Disparar um novo membro pelo simulador.
7. Gerar o evento real escolhido no YouTube e mostrar sua chegada ao sistema.
8. Recarregar/entrar novamente e confirmar que configurações persistiram.
9. Mostrar rapidamente logs, testes e documentação de execução.
10. Explicar em poucos minutos as decisões tomadas, dificuldades encontradas e o que seria feito numa V2.

13. Observações atuais sobre a API do YouTube

Estas observações servem apenas para evitar que a entrega fique presa a comportamentos que a plataforma não garante. Elas não determinam a implementação.

- A YouTube Data API oferece subscriptions.list com myRecentSubscribers para recuperar inscritos recentes do canal autenticado, porém a quantidade e a visibilidade dos resultados podem ser limitadas.
- O endpoint de members existe, mas o acesso é restrito a canais elegíveis/allowlisted e exige escopo específico; portanto, não se deve tornar a aprovação externa uma condição de sucesso do exercício.
- Para live chat, a API oficial disponibiliza leitura de mensagens e um método de streaming de baixa latência (streamList), o que o torna um bom candidato para demonstração de evento real.

Referências técnicas oficiais consultadas em 17/09/2026:

https://developers.google.com/youtube/v3/docs/subscriptions/list

https://developers.google.com/youtube/v3/docs/members

https://developers.google.com/youtube/v3/docs/members/list

https://developers.google.com/youtube/v3/live/docs/liveChatMessages/streamList

Entrega esperada

Repositório com a aplicação funcional, instruções de execução local e uma demonstração da V1. O cliente avaliará principalmente a clareza do fluxo, funcionamento ponta a ponta, qualidade das decisões e capacidade de explicar limitações - não a escolha de framework.